Pre-Deployment Audit — How-To

This runbook is for release engineers, QA leads, and anyone running the pre-deployment audit gate. For scanner architecture see `.kiro/specs/pre-deployment-e2e-audit/design.md`. For requirements see `.kiro/specs/pre-deployment-e2e-audit/requirements.md`.

TL;DR

```
npm run audit -- --pr
```

Exit code 0 → deploy is safe to proceed. Non-zero → deploy is blocked. The full human-readable verdict lives at `audit/output/audit-report.md` and the machine-readable verdict at `audit/output/verdict.json`.

What the gate actually enforces

The audit pipeline runs 11 stages and writes a Go / Go-with-backlog / No-Go verdict based on the severity of findings. The Go/No-Go Matrix is the single source of truth:

Blocker	Critical	Major	Verdict
≥ 1	any	any	No-Go
0	≥ 1 unwaived	any	No-Go
0	≥ 1 waived	any	Go-with-backlog
0	0	> 0	Go-with-backlog
0	0	0	Go

Minor and Trivial findings never block deploy.

Running the audit locally

Standard PR-mode run

```
npm run audit -- --pr
```

Runs in ~90 seconds on a Windows laptop. Covers: lint, tsc, property tests, security scans, design-token enforcement, i18n parity, perf budget, and the a11y baseline. Skips the infra-dependent stages (connectivity, RLS matrix, cron health, full E2E) that need a staging Supabase project.

Single-stage run

Useful when iterating on a specific finding:

```
npm run audit -- --stage=security
npm run audit -- --stage=a11y
npm run audit -- --stage=perf
```

Skipping a stage

When the stage is already known-green (e.g., you just ran the build):

```
npm run audit -- --pr --skip=build
```

Reading the audit report

After any run, `audit/output/audit-report.md` is regenerated. It always contains these sections in order:

1. **Header** with verdict, commit SHA, migration head, environment ID
2. **Executive Summary** with severity counts
3. **Per-Stage Status** showing which stages passed/failed/skipped and their durations
4. **Aggregator Warnings** — only present if an artifact was malformed
5. **Findings** grouped by severity from Blocker to Trivial
6. **Go/No-Go Matrix** (reference)
7. **Provenance** listing every finding artifact ingested

Triaging findings by severity

🛑 Blocker

Deploy is forbidden. No waiver possible. Fix the code and re-run the gate. Examples:

- Service-role JWT leaked into the client bundle
- Admin mutation missing `logAuditEvent` (audit trail gap)
- Unauthorized `VITE_*` env var references
- TypeScript compilation errors
- Lint errors (zero-warning policy)

🔴 Critical

Blocks deploy unless a signed, time-bounded waiver is recorded. See Requesting a waiver below. Examples:

- Any Role's critical-path E2E test failing
- Unfiltered realtime subscription on a tenant-scoped table
- RLS matrix negative-test failure
- Cron endpoint unreachable

🟠 Major

Deploy proceeds as "Go-with-backlog". The finding must be tracked but does not block the current release. Examples:

- Bundle size grew past the 110% cap
- Physical `ml-4` margin without a logical `ms-4` counterpart (RTL)
- Missing `aria-label` on an icon-only button
- en↔ar locale key parity drift
- Unfiltered realtime subscription on a non-tenant-scoped table
- List page without pagination

🟡 Minor

Cosmetic or convenience issue with a clear user workaround. Tracked but does not affect deploy. Examples:

- Known runner flake (vitest#4562 Windows fork-pool crash)
- Expired exception entry (scanner auto-demotes to Minor)

◯ Trivial

Documentation or stylistic nit. No action required. Examples:

- Bundle baseline not yet frozen (first-run advisory)

Requesting a waiver

Critical findings can be waived for a time-bounded release window when three signers approve. The waiver file is **gitignored** so it never leaks signed approvals into the repo.

1. Copy the template:

   ```
   cp audit/waivers.example.json audit/waivers.json
   ```

2. Edit `audit/waivers.json` and fill in:
   - `findingId` — copy the ID from `audit-report.md`
   - `signers.releaseEngineer` / `signers.qaLead` / `signers.techLead` — email addresses of the three approvers
   - `expiresAt` — ISO 8601 timestamp when the waiver expires (typically 2–4 weeks out; never more than one quarter)
   - `rationale` — 1–3 sentences on why the Critical is acceptable to ship and how the underlying fix is tracked (ticket ID required)
3. Re-run `npm run audit -- --pr`. The verdict should flip from No-Go to Go-with-backlog if every Critical has a matching valid waiver.
4. Never commit `audit/waivers.json`. The deploy runbook consumes it out-of-band (the release engineer shows the signed file to the QA lead and tech lead, records its SHA-256 in the release ticket, then discards it after the release).

A waiver is **invalid** when:

- Any signer field is missing or empty
- `expiresAt` is in the past (time-bounded scope is non-negotiable)
- Severity is not exactly `Critical` (Blockers cannot be waived)

Invalid waivers are silently rejected by the aggregator; the Critical re-fails the gate as if no waiver existed.

Adding or updating a baseline

Baselines are committed JSON files under `audit/baselines/`. Each carries `createdAt` and `lockedByCommit` so changes are reviewable. To update:

1. Run the audit locally with the new code in place.
2. Confirm the new measurement is the intended value.
3. Edit the relevant baseline file:
   - Set `createdAt` to the current ISO timestamp.
   - Set `lockedByCommit` to the commit SHA that justified the change.
   - Update the measured value (e.g., `totalGzippedBytes`).
4. Open a PR dedicated to the baseline update with a rationale in the body. Baseline churn in a feature PR is almost always a smell.

Adding an exception

Some scanners (audit-log coverage, pagination) accept per-file exceptions when the scanner's default rule over-fires on a legitimate pattern. See:

- `audit/baselines/audit-log-coverage-exceptions.json` — mutations that don't require an admin audit log (student-scoped content, etc.)
- `audit/baselines/pagination-exceptions.json` — list pages where the result set is bounded by user scope

Each exception entry needs:

- `file` — POSIX-style relative path
- `rationale` — human-readable justification
- Optional `expiresAt` — after which the scanner auto-demotes the finding to Minor (keeps the backlog honest)

Debugging a failing stage

The audit manifest at `audit/output/manifest.json` records which stage failed. Each stage also writes its own `*-findings.json` with the full finding list — open that file to see exact file paths, line numbers, and reproduction detail.

For the process stages (lint, tsc, propertyTests, build), the stdout and stderr tails are captured inside the finding's `detail` block. Look there first before re-running the underlying command.

CI integration

The gate runs as the `Audit` job in `.github/workflows/ci.yml`. PR mode is the default; pre-deploy mode runs on tag pushes. The workflow uploads `audit/output/audit-report.md` as a build artifact so the GitHub UI surfaces the report without needing a local run.

Related docs

- `audit-fixtures-deploy.md` — deploy runbook for the staging-only fixture Edge Function
- Spec design — architecture of the audit pipeline
- Spec requirements — EARS requirements each stage satisfies